UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL

FLUTTER



JULIO 2026-DICIEMBRE 2026

16/10/2026

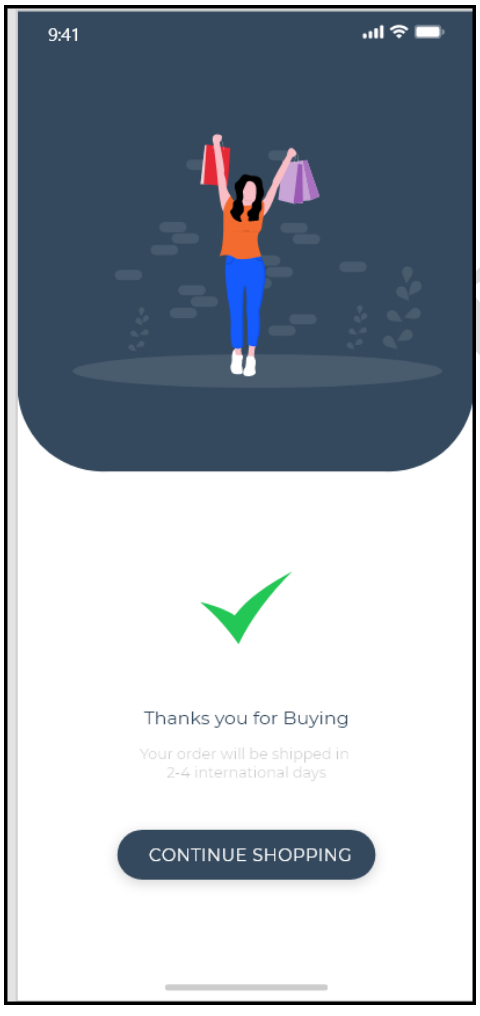
Ejercicio # 1

En esta práctica vamos a iniciar con el diseño de la **tercera pantalla** para finalizar la compra de la aplicación en Flutter.

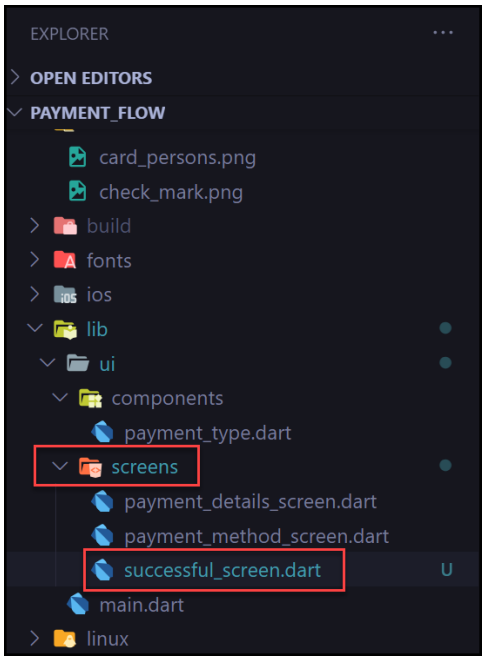
Clay Aldás

Finalizar compra #1

1. En esta práctica vamos a crear la **última pantalla** del proyecto. Para empezar, abra el **mockup** en **Adobe XD**.



2. En Visual Studio Code, **seleccione** la carpeta **screens**, dentro de esta cree el archivo con el nombre **successful_screen.dart** para realizar el diseño de la pantalla para finalizar la compra.



3. En este **archivo** cree la clase **SuccessfulScreen**. Para esta pantalla **no es necesario** guardar **ningún estado** por lo tanto **herede** de la clase **StatelessWidget**, importe el paquete de **material**, cree el **constructor constante** con el parámetro **key**, **implemente** el método **build** y dentro de este **regrese** un widget de tipo **Scaffold** para la estructura de la pantalla. El código se muestra a continuación.

successful_screen.dart U

```dart

import 'package:flutter/material.dart';

class SuccessfulScreen extends StatelessWidget {
  const SuccessfulScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return const Scaffold();
  }
}
```

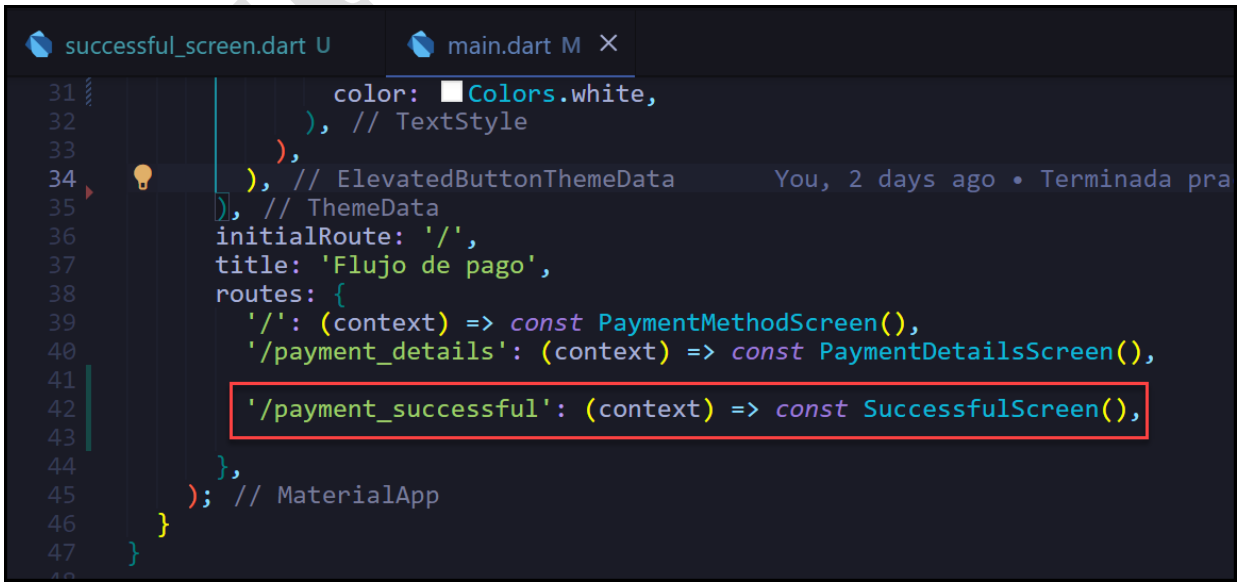
Nota	• Si no sabe cómo generar el **código anterior,** revise una de las prácticas anteriores para poder generar el código con la ayuda de **Visual Studio Code.**

4. Ahora **abra** el archivo **main.dart**, al final de este archivo, en la propiedad **routes** cambie la última llamada de la pantalla **PaymentDetailsScreen** por la nueva pantallla **SuccessfulScreen** creada en el punto anterior, tal y como se muestra a continuación.

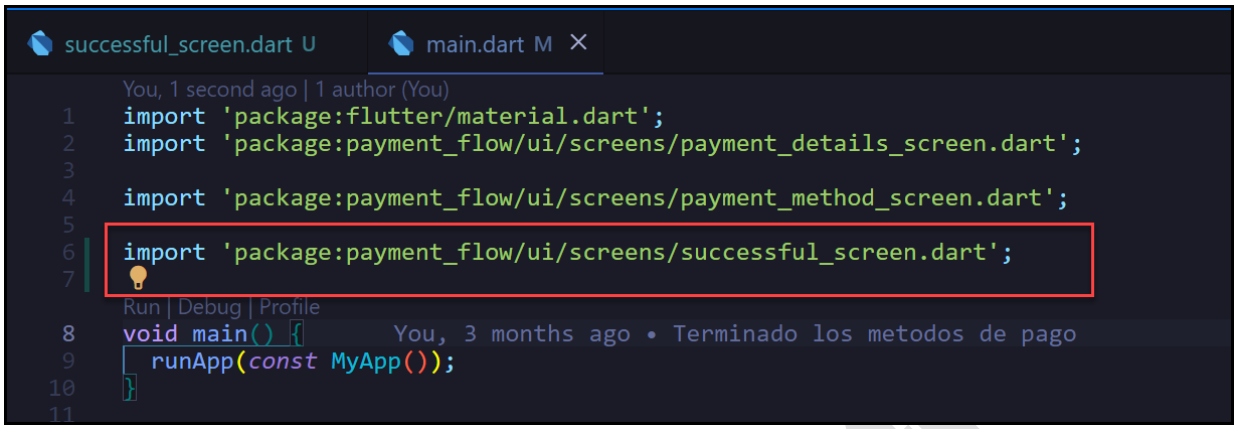
Anterior código antes del cambio:



Nuevo código luego del cambio.



5. Verifique que al **inicio** del archivo **main.dart** se haya **importado** el archivo donde esta definida la pantalla **SuccessfulScreen**. No se importo automáticamente entonces agregue la línea para la importación manualmente.



6. Ahora es **necesario pasar los parámetros** de la segunda pantalla (**PaymentDetailsScreen**) donde estan **los datos de la compra**, hacia la tercera pantalla (**SuccessfulScreen**) que es la pantalla final del proceso.

7. Cuando **utilizamos el navegador** del móvile y creamos una **nueva instancia** de la **ruta** para poder instanciar el wigdet/clase **pantalla** que se va a mostrar, facilmente se puede pasar atributos en el **constructor** de la pantalla a mostrar, pero cuando se están **utilizando rutas** como es el caso del ejemplo para la práctica, no podemos pasar los parámetros de esta forma, sino que hay que pasarlo con **parámetros pero con otro procedimiento**,

8. Por lo tanto vamos a **recibir los parámetros** de la segunda pantalla (**PaymentDetailsScreen**) en la tercera pantalla (**SuccessfulScreen**) de otra forma, en este caso la información que se requiere pasar entre pantallas son los **datos de la tarjeta ingresados en la pantalla**.

9. Para esto **se necesita una clase** que se pueda **instanciar** para pasar los **parámetros** necesarios. Por lo tanto, dentro del archivo **successful_screen.dart** al final del archivo **cree la clase** llamada **MyArguments** que se utilizará para este objetivo, esto se indicada a continuación en el código.

successful_screen.dart U

```dart
import 'package:flutter/material.dart';

class SuccessfulScreen extends StatelessWidget {
  const SuccessfulScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return const Scaffold();
  }
}

class MyArguments {

}
```

10. Los parámetros que se necesita pasar entre pantalllas son algunos de los datos que tenemos en la segunda pantlla, exactamente los que algunos que están en la clase **_PaymentDetailsScreenState**, clase que se encuentra en el archivo **payment_details_screen.dart**, algunos de estos datos se muestran en la siguiente figura (no se van a pasar todos los datos como parámetros, **únicamente algunos para el ejemplo**).

successful_screen.dart U | payment_details_screen.dart

```dart
}
}

You, 2 days ago | 1 author (You)
class _PaymentDetailsScreenState extends State<PaymentDetailsScreen> {
  final formKey = GlobalKey<FormState>();

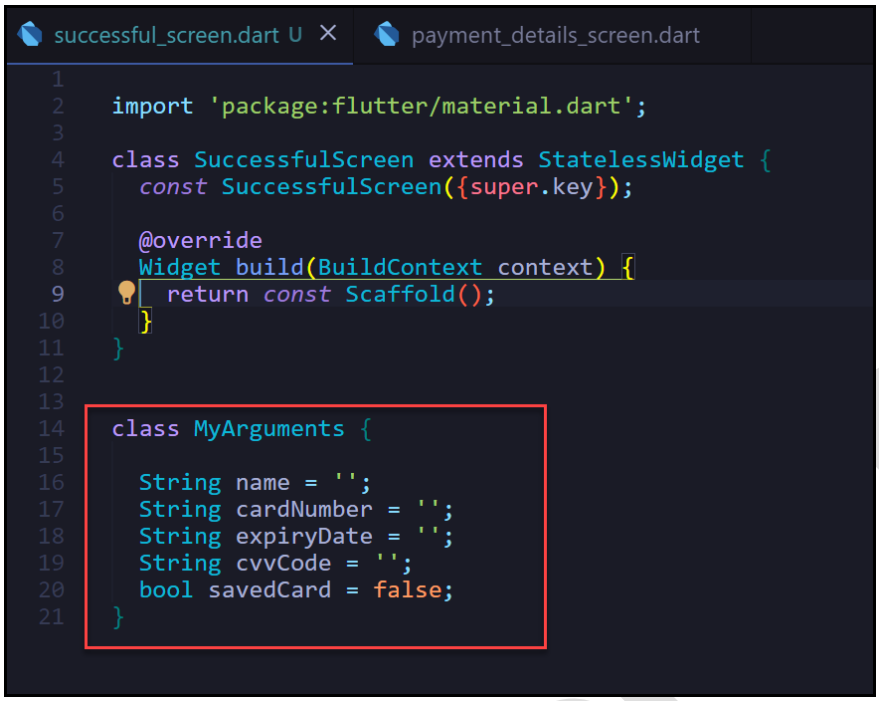
  String name = '';
  String cardNumber = '';
  String expiryDate = '';
  String cvvCode = '';

  late FocusNode cardNumberFocus;
  late FocusNode expiryDateFocus;
  late FocusNode cvvCodeFocus;

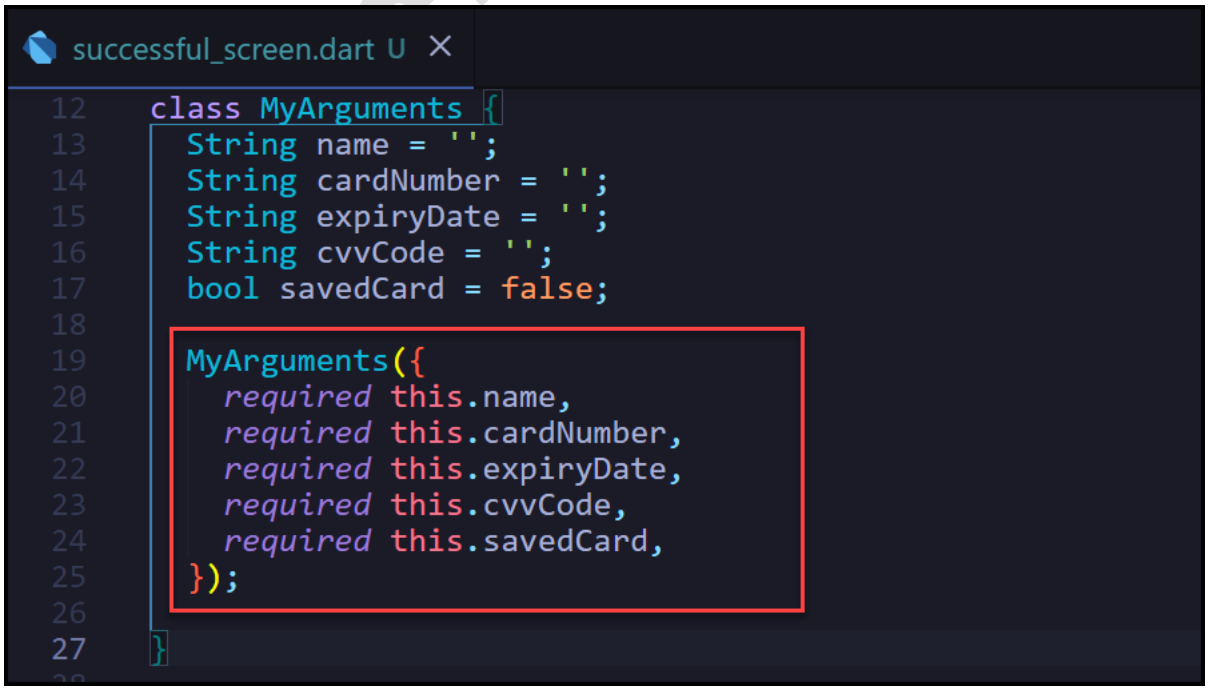
  bool savedCard = false;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(),
```

11. Por lo tanto **copie** las variables que se muestran a continuación (menos las de tipo **FocusNode**) al archivo **successful_screen.dart** dentro de la clase **MyArguments**, el código sería el siguiente.



12. **Cree** un **constructor** y pase como **parámetros todas las variables** copiadias, como se muestra a continuación.



13. Vaya al archivo **payment_details_screen.dart** y dentro de este vaya al método **showSuccessful**, dentro de este **instancie** y **pase los argumentos** a la clase **MyArguments** creada anteriormente, esto antes de mostrar la pantalla, el código necesario se muestra a continuación.

```dart
payment_details_screen.dart 1, M
258
259   void showSuccessful(BuildContext context) {      You, 31 seconds ago • Uncommitted changes
260     if (formKey.currentState!.validate()) {
261       formKey.currentState?.save();
262
263       MyArguments myArguments = MyArguments(    The value of the local variable 'myArguments'
264           name: name,
265           cardNumber: cardNumber,
266           expiryDate: expiryDate,
267           cvvCode: cvvCode,
268           savedCard: savedCard);
269
270       Navigator.pushNamedAndRemoveUntil(context, '/payment_successful',
271           ModalRoute.withName('/payment_successful'));
272     }
273   }
274 }
275
      You, 2 days ago | 1 author (You)
276 class CustomInputDecoration extends InputDecoration {
277   final String inputText;
278
279   CustomInputDecoration(this.inputText)
```

Nota	• En el código anterior, tanto los nombres de los **parámetros con nombre** de la clase **MyArguments** como los nombres de las variables de la clase **_PaymentDetailsScreenState** que es donde están declarados las variables con la cuales se llama al constructor de la clase **MyArguments**, tienen nombres idénticos, esto facilita la lectura del código.

14. **Verifique** que se haya **importado** al inicio del archivo anterior el archivo **successful_screen.dart** que es donde esta definida la clase **MyArguments**.

```dart
// successful_screen.dart U | payment_details_screen.dart 1, M
// You, 4 minutes ago | 1 author (You)
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import 'package:font_awesome_flutter/font_awesome_flutter.dart';
import 'package:mask_text_input_formatter/mask_text_input_formatter.dart';

import 'package:payment_flow/ui/screens/successful_screen.dart';

// You, 3 months ago | 1 author (You)
class PaymentDetailsScreen extends StatefulWidget {
  const PaymentDetailsScreen({super.key});

  @override
  State<PaymentDetailsScreen> createState() {
    return _PaymentDetailsScreenState();
  }
}
```

15. Dentro del método **showSuccessful** ahora **pase** como **argumento** la variable llamada **myArguments** creada anteriormente en el método **pushNamedAndRemoveUntil**, de la clase **Navigator**, para que antes que se muestre pantalla indicada se pase los datos requeridos, esto se muestra en la siguiente figura.

```dart
// payment_details_screen.dart M
  void showSuccessful(BuildContext context) {      You, 1 second ago • Uncommitted changes
    if (formKey.currentState!.validate()) {
      formKey.currentState?.save();

      MyArguments myArguments = MyArguments(
          name: name,
          cardNumber: cardNumber,
          expiryDate: expiryDate,
          cvvCode: cvvCode,
          savedCard: savedCard);

      Navigator.pushNamedAndRemoveUntil(context, '/payment_successful',
          ModalRoute.withName('/payment_successful'), arguments: myArguments);
    }
  }
}

// You, 2 days ago | 1 author (You)
class CustomInputDecoration extends InputDecoration {
  final String inputText;

  CustomInputDecoration(this.inputText)
```

16. Regrese al archivo **successful_screen.dart** y en la clase **MyArguments** sobrescriba el método **toString** para que este muestre los datos de la clase la utilizar este método. El código se muestra en la siguiente figura.

successful_screen.dart U

```dart
class MyArguments {
  String name = '';
  String cardNumber = '';
  String expiryDate = '';
  String cvvCode = '';
  bool savedCard = false;

  MyArguments({
    required this.name,
    required this.cardNumber,
    required this.expiryDate,
    required this.cvvCode,
    required this.savedCard,
  });

  @override
  String toString() {
    return 'MyArguments: {"$name, $cardNumber, $expiryDate, $cvvCode, $savedCard}';
  }
}
```

17. **Ejecute** y **pruebe** el proyecto.

18. Si se **ingresa los datos** en la pantalla de **Detalles de Pago** y luego se **presiona** en el botón **CONFIRMAR PAGO**, entonces se instancia la variable **myArguments** y se pasa los **argumentos** (valores) en los **parámetros** del constructor de la claes **MyArguments** hacia la siguiente pantalla (**SuccessfulScreen**), que esta en el archivo **successful_screen.dart**.



19. Ahora en la pantalla **SuccessfulScreen** del archivo **successful_screen.dart** vamos a **recuperar** e **imprimir** los datos (en **consola** para el ejemplo) con la clase **ModalRoute**, con el método **of** dentro del método **build**, los **parámetros enviados** desde la pantalla **PaymentDetailsScreen**, que esta en el archivo **payment_details_screen.dart**. Para esto digite el siguiente código.

successful_screen.dart U

```dart
class SuccessfulScreen extends StatelessWidget {
  const SuccessfulScreen({super.key});

  @override
  Widget build(BuildContext context) {

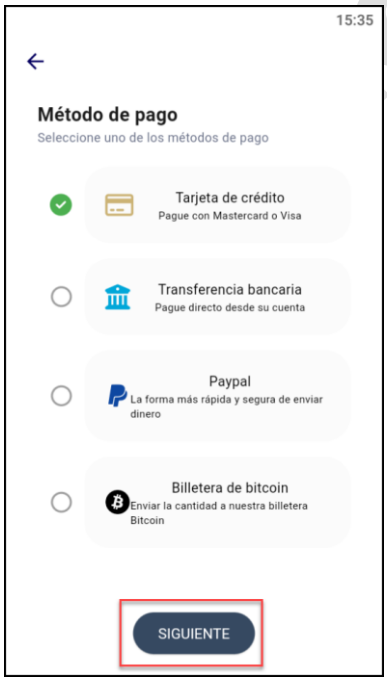
    print(ModalRoute.of(context)?.settings.arguments);    Don't invoke 'print'

    return const Scaffold();
  }
}

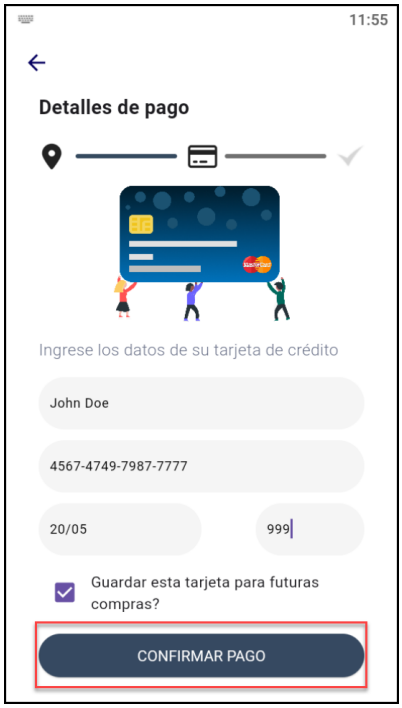
class MyArguments {
  String name = '';
  String cardNumber = '';
  String expiryDate = '';
  String cvvCode = '';
  bool savedCard = false;
```

20. **Ejecute** y **pruebe** la aplicación.

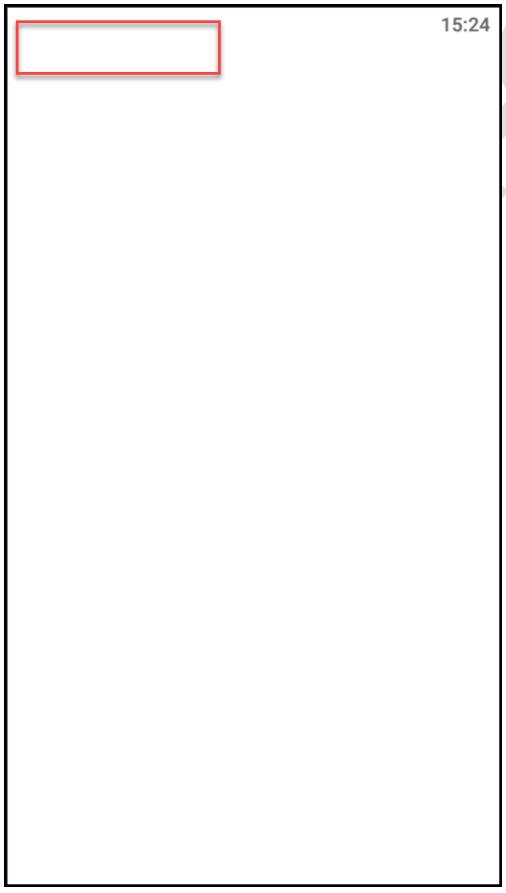
Primera pantalla.



Segunda pantalla. Ingrese datos en la **segunda pantalla**.

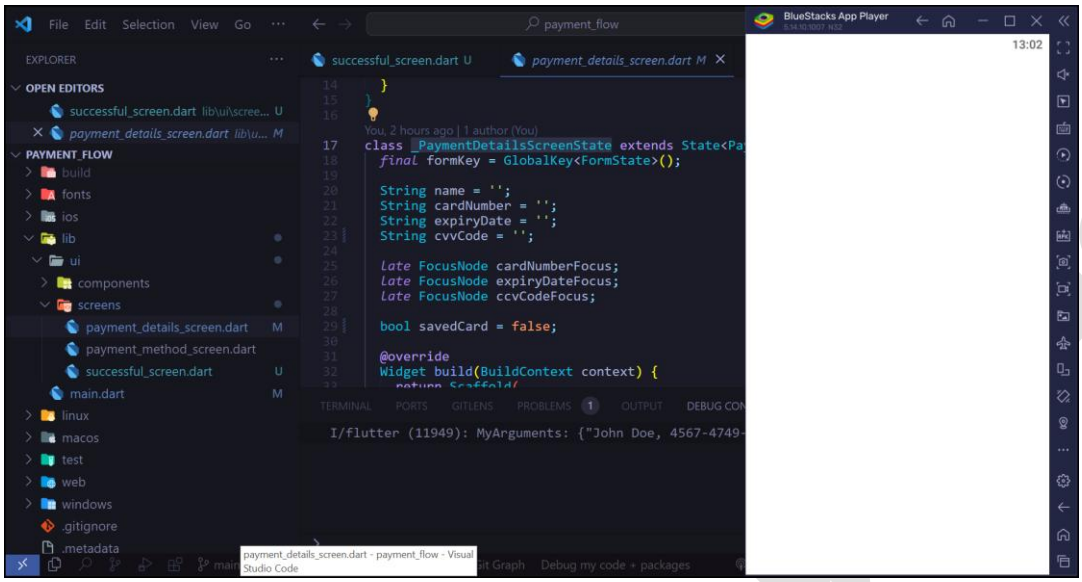


Tercera pantalla luego de presionar el botón **CONFIRMAR PAGO**.



Clay Aldás

Tercera pantalla con la salida por **consola**. Pantalla **DEBUG CONSOLE** de Visual Studio Code con la salida que muestra los datos de la clase **MyArguments** impresos por el método **print**.



21. Luego de **presionar** en el botón **CONFIRMAR PAGO**, se muestra la tercera ventana (**SuccessfulScreen)** y en la Visua Studio Code en la ventana **DEBUG CONSOLE** se puede visualizar la salida del método **print** del **código anterior**.



Nota	• En la **pantalla anterior** que por el momento esta en blanco, no existe el botón para ir **hacia atrás,** esta desactivada la navegación, no hay historial de navegación, se cierra las sesiones abiertas, todo esto **por seguridad** por el tipo de aplicación y pantalla que se está implementando y así evitar que el **usuario cometa errores**. • Por lo tanto, es necesario que se **detenga totalmente la aplicación** y se inicie nuevamente para poder **probar el proyecto con toda la funcionalidad.** • Como se puede apreciar, es posible pasar datos de una pantalla a otra de forma **muy sencilla** aunque la aplicación utilize rutas (**routes**).

flutterClay

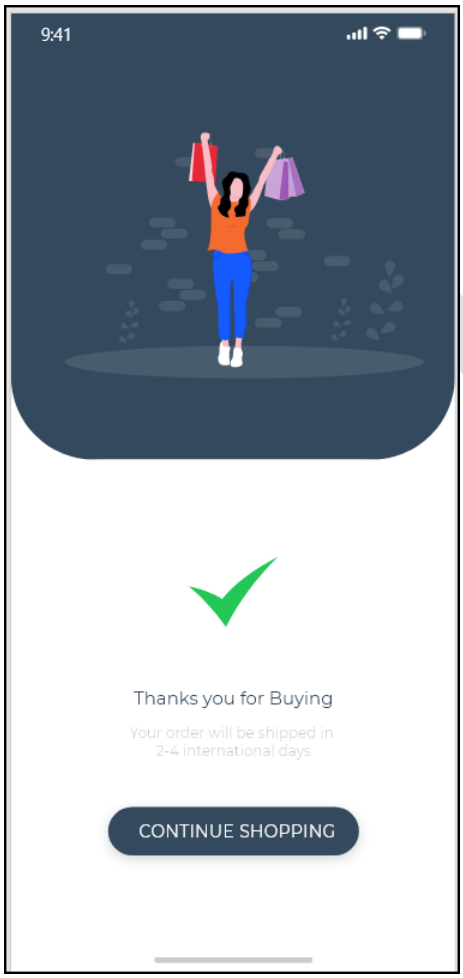
16/10/2026

Ejercicio # 2

En esta práctica vamos a terminar el diseño de la **tercera pantalla** para finalizar la compra de la aplicación en Flutter.

Finalizar compra #2

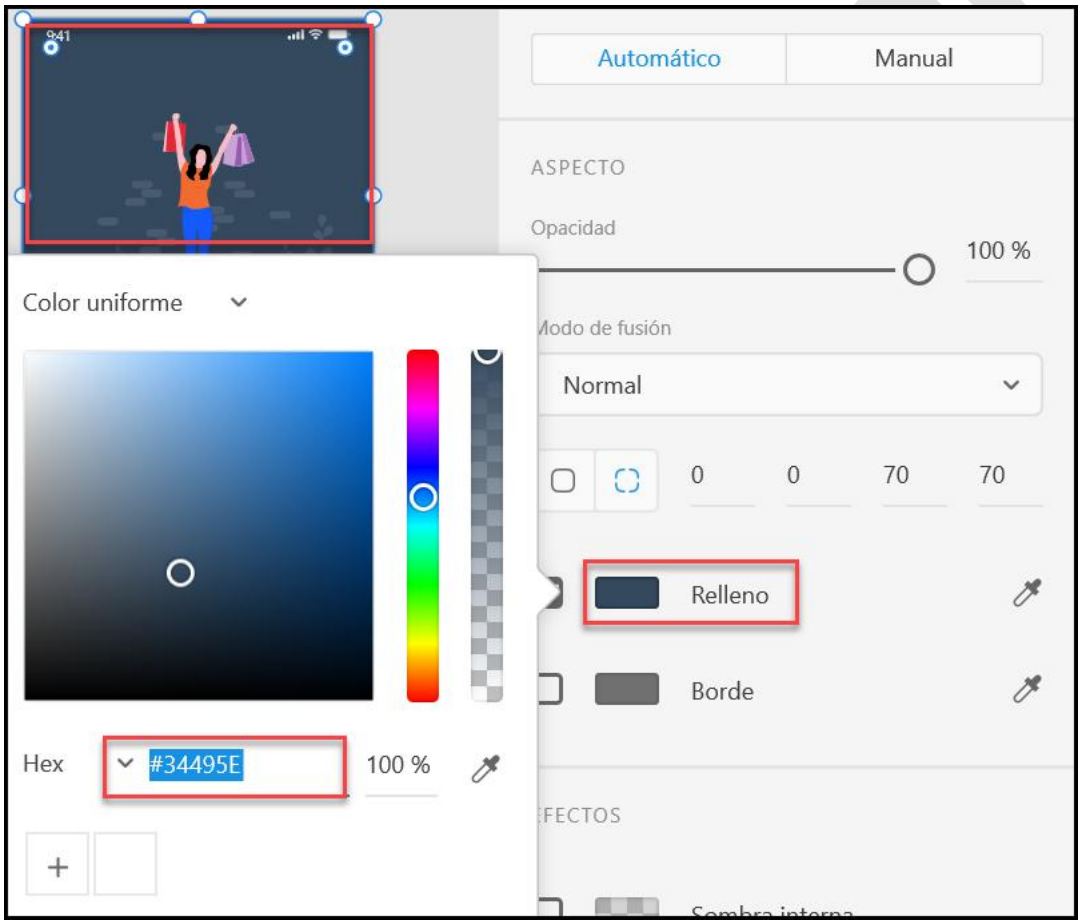
1. Vamos a continuar con el diseño de última pantalla. Para esto **abra** el **mackup** para analizar la pantalla.



Clay Aldás

2. Una forma de diseñar la pantalla sería utilzando por ejemplo un **Column** principal, un **Container**, una **imagen** de fondo, en la parte de abajo puede ser otro **Column**, un **botón** (verde), un **texto**, un **botón**. Esta es solo una forma crear la pantalla, pero se podría utilizar otros widgets diferentes para crear la UI, es decir hay varias formas de abordar la creación de la pantalla.

3. Algo muy importante a resarltar, es que el **color** de fondo exacto para el **Container** se lo obtiene del **mockup**. Para esto abra el **mockup** en **Adobe XD**, **seleccione** alguna parte del **cuadro indicado** y en la propiedad **relleno presione** sobre este y esto hace que se visualice el **color exacto** establecido para el diseño.



Color: **#34495E**

4. Para iniciar abra el archivo **successful_screen.dart**, en la clase **SuccessfulScreen** dentro del método **build**, **comente** o **elimine** la línea con la instrucción **print** utilizada anteriormente.

successful_screen.dart U

```dart
import 'package:flutter/material.dart';

class SuccessfulScreen extends StatelessWidget {
  const SuccessfulScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return const Scaffold();
  }
}

class MyArguments {
  String name = '';
  String cardNumber = '';
  String expiryDate = '';
  String cvvCode = '';
  bool savedCard = false;
```

5. Luego de **eliminar** la línea indicada, para el diseño de la pantalla vamos a **utilizar** como base un widget/clase **Column** y dentro un widdget/clase **Container**. Para esto **digite** el siguiente código en el método **build** dentro del **Scaffold** en la propiedad **body**.

successful_screen.dart M

```dart
  const SuccessfulScreen({super.key});

  @override
  Widget build(BuildContext context) {

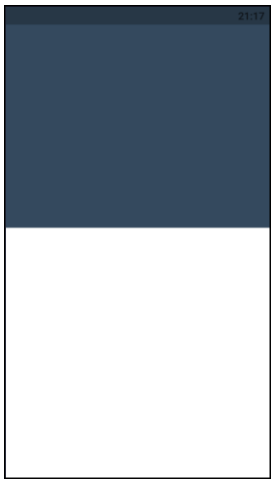
    return Scaffold(

      body: Column(       You, 10 seconds ago • Uncommitted changes
        children: [
          Container(
            height: 300,
            decoration: const BoxDecoration(
              color: Color(0XFF34495E)
            ), // BoxDecoration
          ), // Container
        ],
      ), // Column

    ); // Scaffold
  }
}
```

Nota	• Es muy **importante notar** que en el código anterior el **Scaffold** ya no tiene la palabra reservada **const**. Esto es muy importante indicar, puesto que el código anterior no compilará y por lo tanto generará un **error**.

6. **Ejecute** y **pruebe** el proyecto.



7. Ahora de acuerdo al **mockup**, hay que establecer los **bordes redondeados** para el **Container**, pero únicamente en la **parte inferior** de este widget/clase. Por lo tanto cree el siguiente código.

successful_screen.dart M

```
const SuccessfulScreen({super.key});

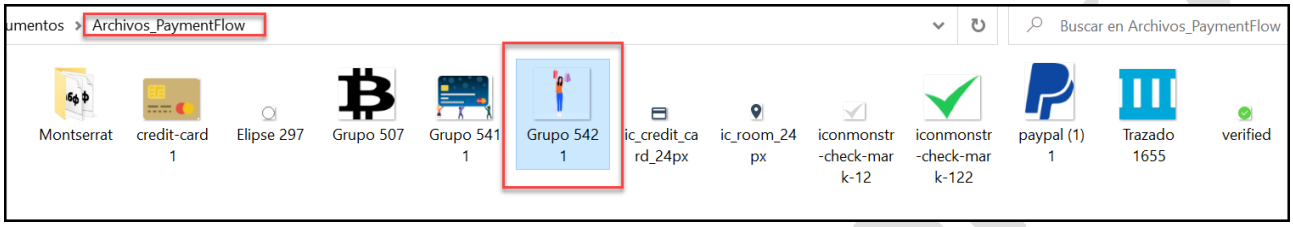
@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Column(
      children: [
        Container(
          height: 300,
          decoration: const BoxDecoration(
            color: Color(0XFF34495E),

            borderRadius: BorderRadius.only(        You, 3 seconds ago • Uncommitted changes
                bottomLeft: Radius.circular(60),
                bottomRight: Radius.circular(60)), // BorderRadius.only

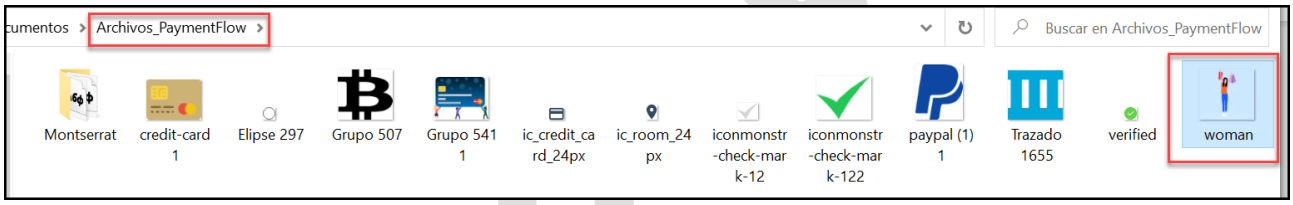
          ), // BoxDecoration
        ), // Container
      ],
    ), // Column
  ); // Scaffold
}
}
```

8. De acuardo al mockup, a continuación se necesita agregar una **imagen** a la pantalla. Para esto vaya a la carpeta "**Archivos PaymentFlow**", donde se guardo las imágenes en la **primera práctica** y seleccione el archivo **Grupo 542 1.PNG**, **renombre** el archivo por el nombre **woman.png** y **copielo** dentro de la carpeta **assets** del proyecto.

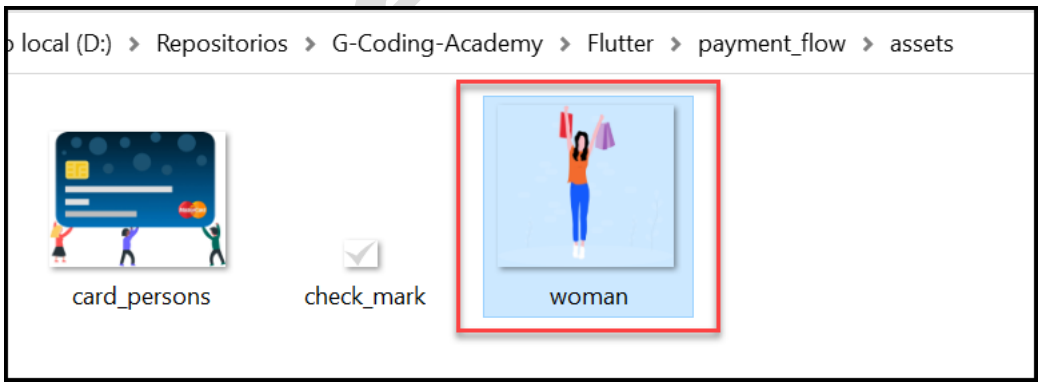
Carpeta de trabajo **Archivos_PaymentFlow**.



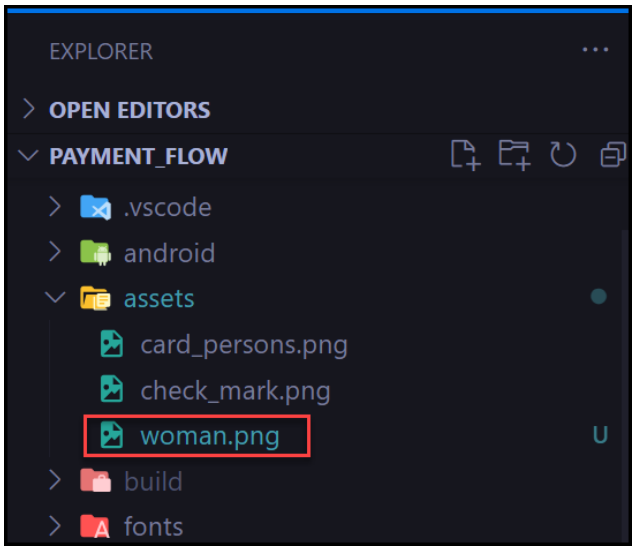
Achivo **renombrado**.



El archivo **woman.png** se copia a la carpeta **assets** del proyecto.

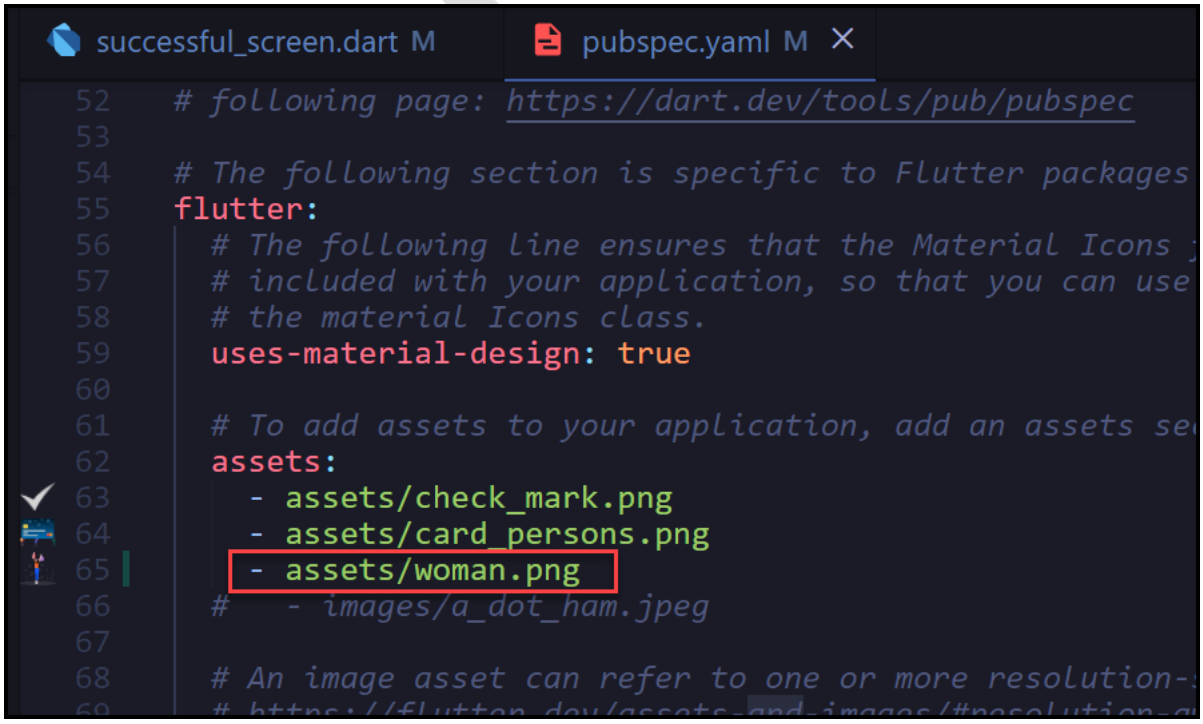


Archivo **woman.png** se muestra en **Visual Studio Code**.



Nota	• Si no tiene el archivo **Grupo 542 1.PNG** con la extensión adecuada, o no existe en la carpeta de trabajo, revise las **primeras prácticas** y utilice **Adobe XD** y **Figma** para obtener el archivo en el formado correcto para la práctica.

9. Abra el archivo **pubspec.yaml**, en la sección **assets** agregue la entrada para el archivo **woman.png** a utilizar.



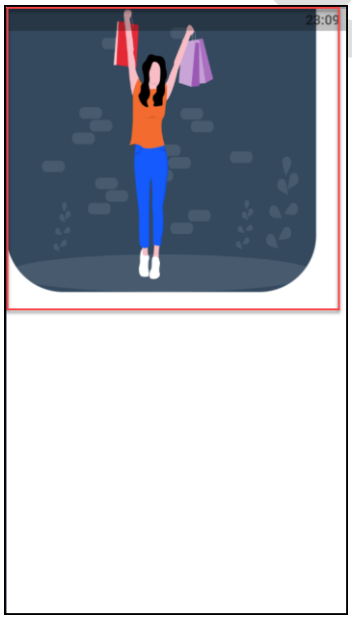
10. Abra el archivo **successful_screen.dart**, en la clase **SuccessfulScreen**, dentro del método **build**, dentro del **container**, agregue un widget/clase **Image** en la propiedad **child** para referenciar la **imagen anterior** agregada.

```dart
Widget build(BuildContext context) {
  return Scaffold(
    body: Column(
      children: [
        Container(      You, 1 minute ago • Uncommitte
          height: 300,
          decoration: const BoxDecoration(
            color: Color(0XFF34495E),
            borderRadius: BorderRadius.only(
                bottomLeft: Radius.circular(60),
                bottomRight: Radius.circular(60)), //
          ), // BoxDecoration

          child: Image.asset('assets/woman.png'),

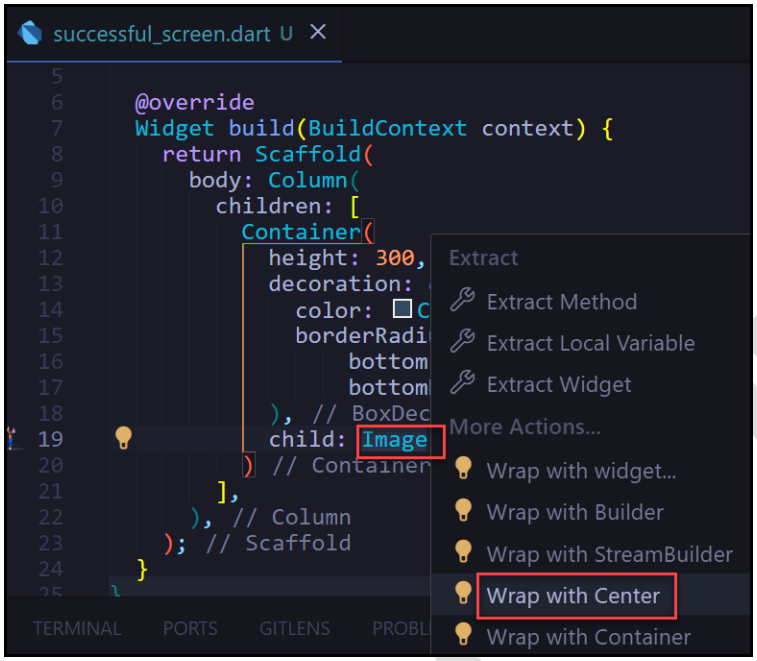
        ), // Container
      ],
    ), // Column
  ); // Scaffold
}
}
```

11. Si **ejecuta** el proyecto se puede apreciar que la **imagen es más grande** que la del **mockup** y se **superpone** con el tamaño del **Container**.

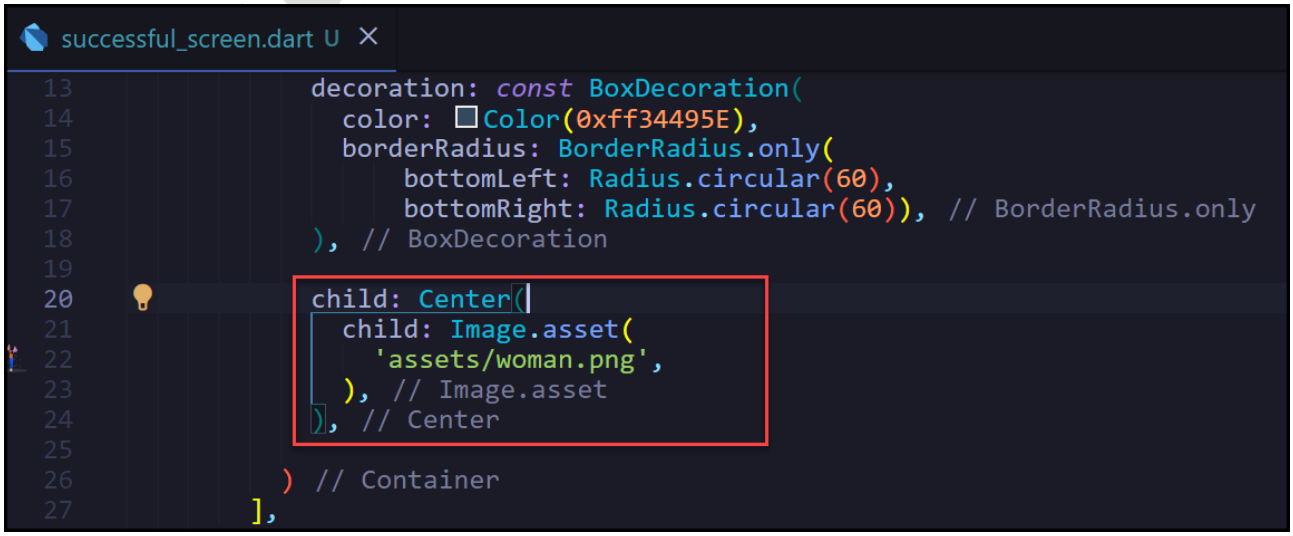


12. Para solucionar este inconveniente, una forma de arreglarlo es utilizar un widget/clase **Center** para **centrar** la **imagen** dentro del **Container**, además es necesario **reducir el tamaño** de la **imagen** porque es muy grande con relación a la del **mockup**. Por lo tanto **envuelva** el widget/clase **Image** dentro de un widget/clase **Center**. Para esto **ponga el cursor** en la palabra **Image**, presione las teclas **CONTROL + PUNTO** (Control + .), de la ventana emergente **seleccione** la opción **Wrap with Center**.

Ponga el **cursor** en cualquier letra de la palabra **Image**. **Seleccione** la opción indicada.



El widget/clase **Image** se envuelve dentro de un widget/clase **Center**.



13. Agregue un tamaño (**height**) para la **imagen**.

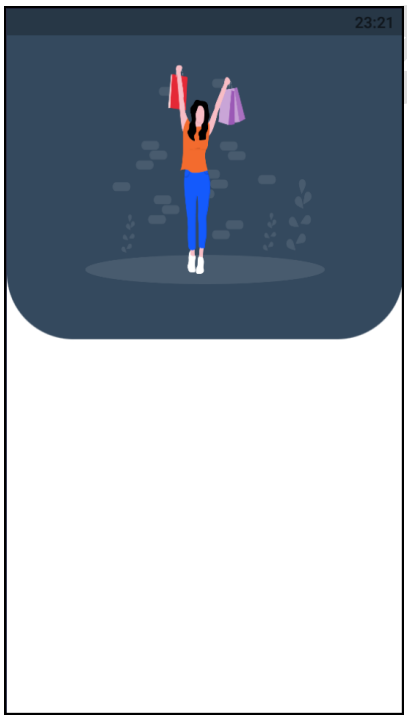
successful_screen.dart M pubspec.yaml M

```
Widget build(BuildContext context) {
  return Scaffold(
    body: Column(
      children: [
        Container(
          height: 300,
          decoration: const BoxDecoration(
            color: Color(0XFF34495E),
            borderRadius: BorderRadius.only(
                bottomLeft: Radius.circular(60),
                bottomRight: Radius.circular(60)), // BorderRadius.only
          ), // BoxDecoration

          child: Center(      You, 39 seconds ago • Uncommitted changes
            child: Image.asset(
              'assets/woman.png',
              height: 200,
            ), // Image.asset
          ), // Center

        ), // Container
      ],
    ), // Column
  ); // Scaffold
}
}
```

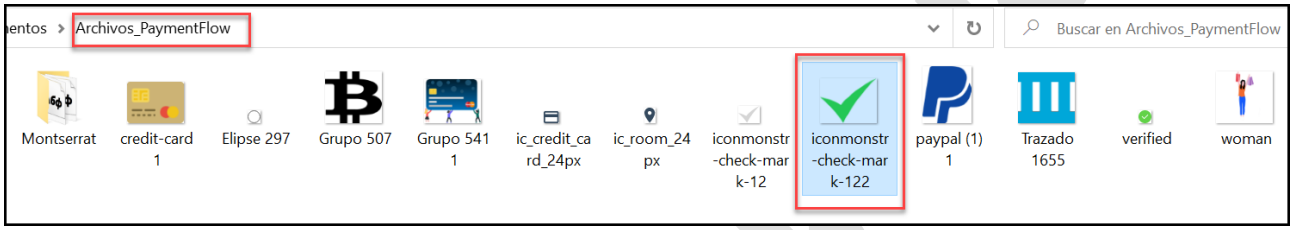
14. **Ejecute** y **pruebe** proyecto.



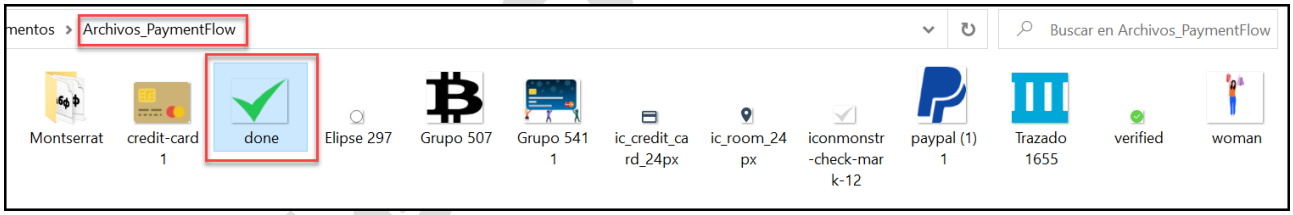
15. Como se puede apreciar ahora la **imagen** esta centrada y el tamaño de esta se parece a la del **mockup**.

16. Repita el proceso anterior, por lo tanto, vaya a la carpeta de trabajo del proyecto "**Archivos_PaymentFlow**", donde se guardo las imágenes en la **primera práctica** y **seleccione** el archivo **iconmonstr-check-mark-122.png** (o como haya nombrado el archivo), **renombre** el archivo por el nombre **done.png** y **copielo** dentro de la carpeta **assets** del proyecto.

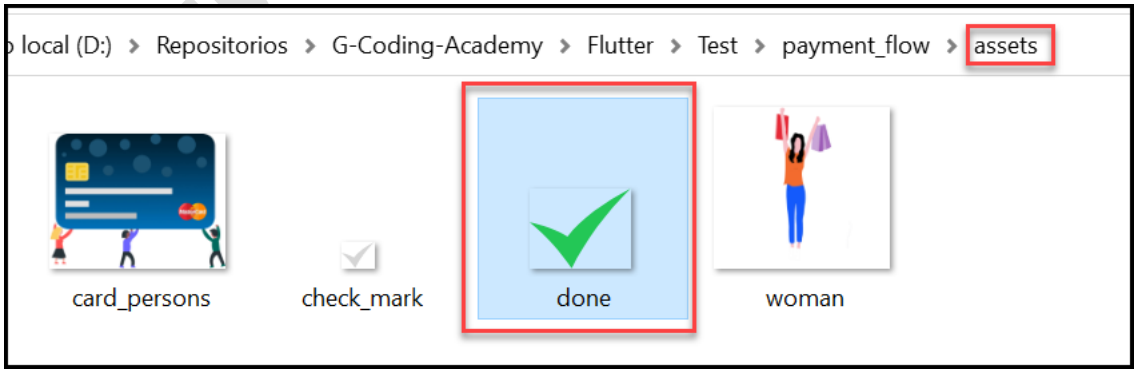
Carpeta de trabajo **Archivos_PaymentFlow**.



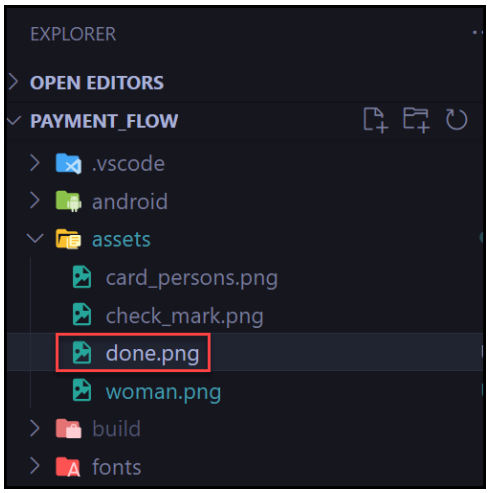
Achivo **renombrado**.



El archivo **done.png** se copia a la carpeta **assets** del proyecto.

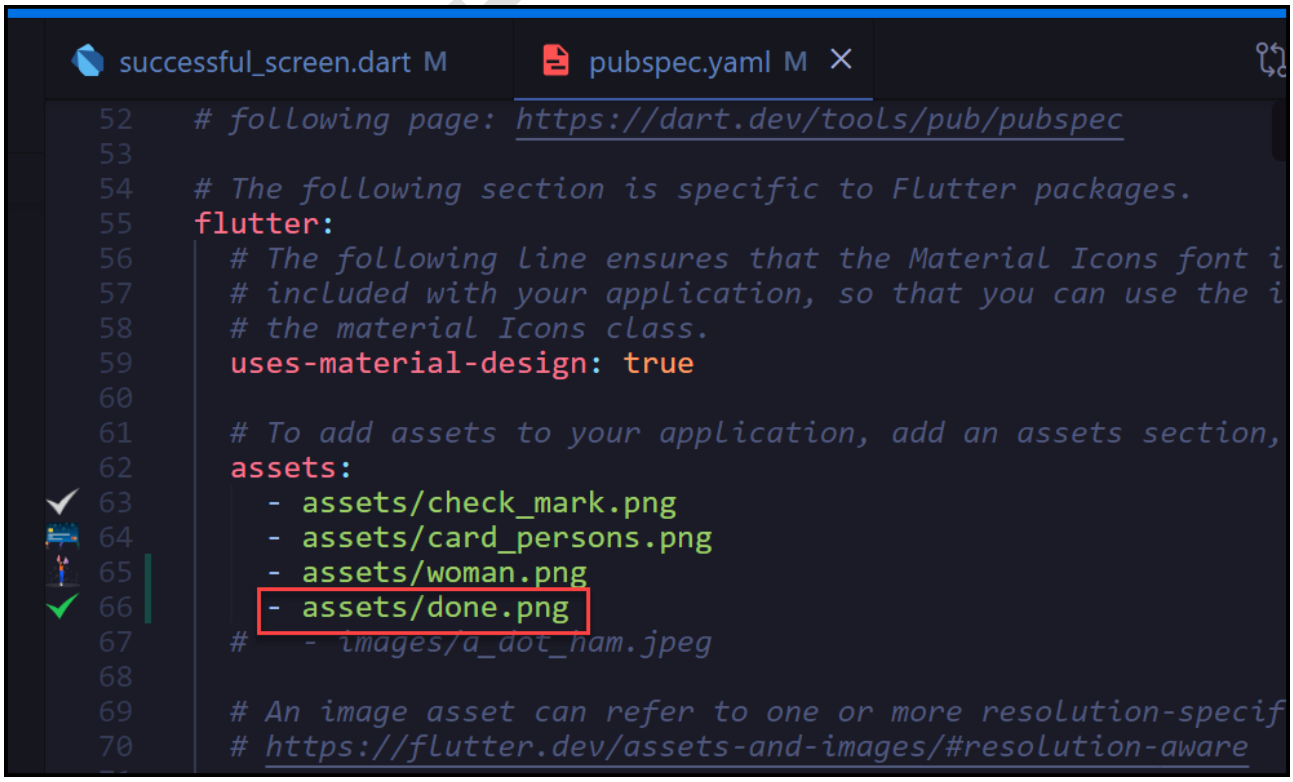


Archivo **done.png** se muestra en Visual Studio Code.

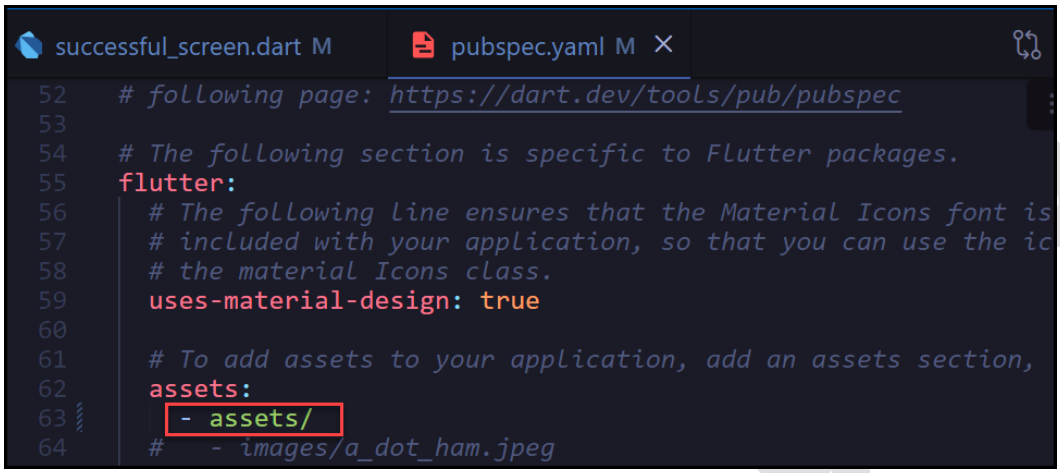


Nota	• Si no tiene el archivo **iconmonstr-check-mark-122.png** con la extensión adecuada, o no existe en la carpeta de trabajo, revise las **primeras prácticas** y utilice **Adobe XD** y **Figma** para obtener el archivo en el formado correcto para la práctica.

17. Abra el archivo **pubspec.yaml**, en la sección **assets** agregue la entrada para el archivo **done.png** copiado anteriormente.



18. Algo muy importante que menicionar, en lugar de **agregar cada archivo de imagen** que se **requiere utilizar** en una aplicación, es mejor utilzar la siguiente entrada en el archivo **pubspec.yaml**, de esta forma **se reconoce cualquier archivo** que se ubique dentro de la carpeta donde están las **imágenes**, en este caso la carpeta **assets**, la entrada necesaria en el archvo **pubspec.yaml** se indica a continuación.



19. Continuamos con **diseño** de la pantalla, con un **Sizebox** **agregue** un espacio **luego** del **Container** para separar **nueva imagen** que se va a adicionar más adelante, luego del **Container**.



20. Luego del **SizeBox** añada un widget/clase **Image** para mostrar la **imagen** llamada **done.png** en la pantalla, y luego **agregue** otro **SizeBox** para dejar un espacio para el próximo componente.

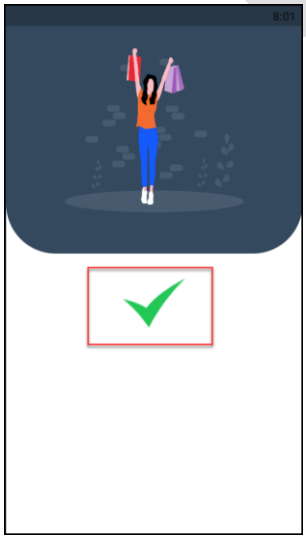
successful_screen.dart M

```
Widget build(BuildContext context) {
  return Scaffold(
    body: Column(
      children: [      You, 17 seconds ago • Uncommitted changes
        Container(
          height: 300,
          decoration: const BoxDecoration(
            color: Color(0XFF34495E),
            borderRadius: BorderRadius.only(
                bottomLeft: Radius.circular(60),
                bottomRight: Radius.circular(60)), // BorderRadius.only
          ), // BoxDecoration
          child: Center(
            child: Image.asset(
              'assets/woman.png',
              height: 200,
            ), // Image.asset
          ), // Center
        ), // Container
        const SizedBox(
          height: 30,
        ), // SizedBox

        Image.asset('assets/done.png', height: 60,),
        const SizedBox(height: 30,)

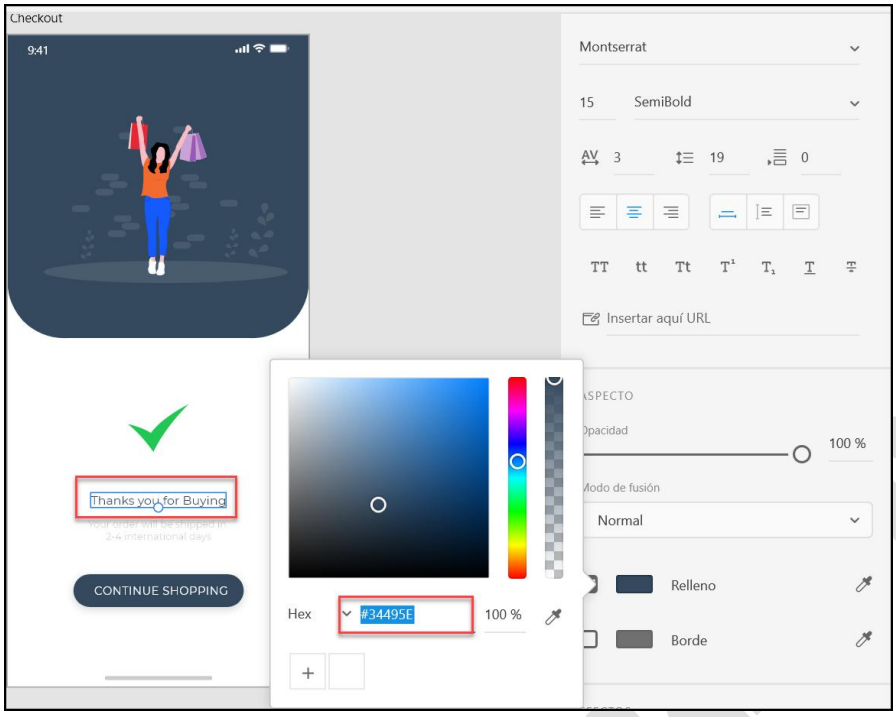
      ],
    ), // Column
  ); // Scaffold
}
}
```

21. **Ejecute** y **pruebe** el proyecto.

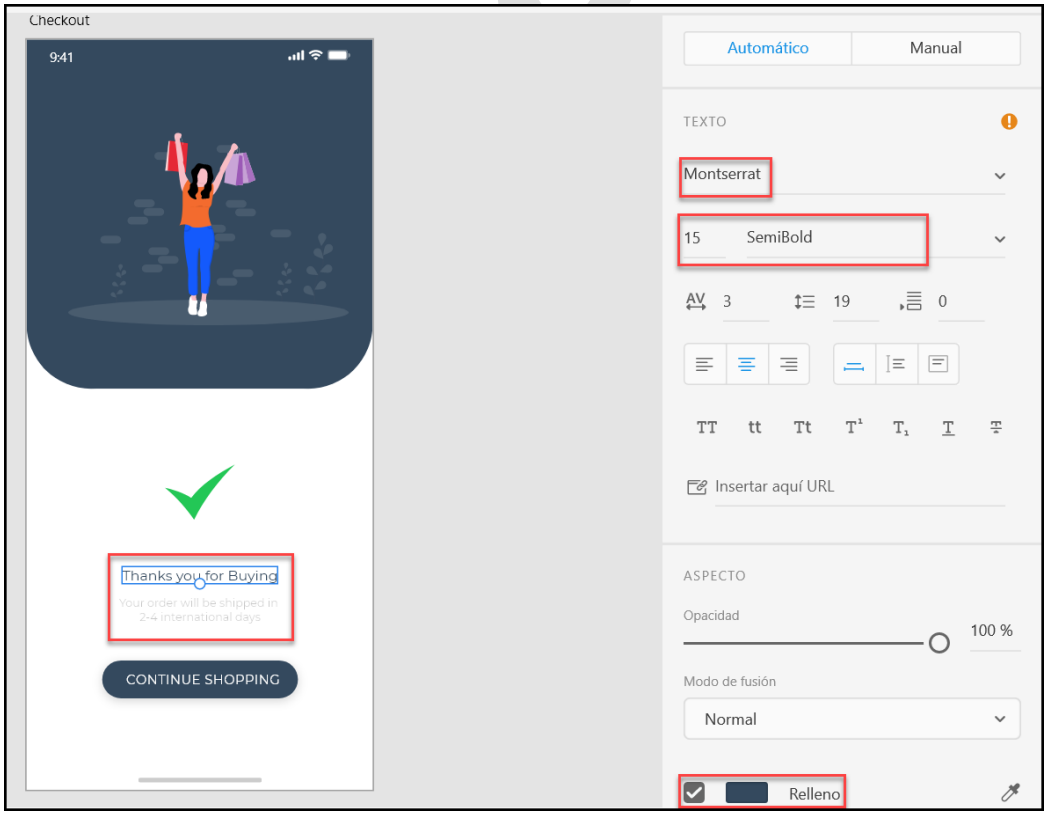


22. Abra el **mockup** en Adobe XD con la tercera pantalla.

Color de la **fuente**.



Más **propiedades** y sus correspondientes valores.



Del **mockup**, de los textos señalados anteriormente se pueden obtener los siguientes datos. Para el **primer texto**:

PROPIEDAD	**COLOR**
Relleno	#34495E
Tamaño	15
Peso	SemiBold Este valor exacto **no existe** en Flutter, el que más se parece es el valor: **FontWeight.w300**
Fuente	Montserrat Esto valor ya se ha ubicado como parte del **Theme** para todo el **proyecto**.

Para el **segundo texto**:

PROPIEDAD	**COLOR**
Relleno	#000000
Tamaño	12
Peso	SemiBold Este valor exacto **no existe** en Flutter, el que más se parece es el valor: **FontWeight.w300**
Fuente	Montserrat Esto valor ya se ha ubicado como parte del **Theme** para todo el **proyecto**.

23. El siguiente paso es agregar un widget/clase **Text** (texto), un **color** (obtenido del **mockup**, con **Adobe XD**), un **tamaño** y un **peso**. Luego añada **otro texto** con **color gris**, y las **mismas propiedades** que el anterior texto pero con el **texto centrado**, esto se muestra a continuación.

FlutterClay Aldás

successful_screen.dart U

```
      child: Image.asset(
        'assets/woman.png',
      ), // Image.asset
    ), // Center
  ), // Container
  const SizedBox(height: 30),
  Image.asset('assets/done.png', height: 60),
  const SizedBox(height: 30),

  const Text(
    'Gracias por su compra',
    style: TextStyle(
        color: Color(0xff34495E),
        fontSize: 15,
        fontWeight: FontWeight.w300), // TextStyle
  ), // Text

  const Text(
    'Su orden será entregada en \n 2-4 días laborables.',
    style: TextStyle(
      color: Color(0xff000000),
      fontSize: 12,
      fontWeight: FontWeight.w300,
    ), // TextStyle
    textAlign: TextAlign.center,
  ), // Text

47 ],
```

Nota	• El tipo de fuente "**Monserrat**" ya se ha agregado como parte del **Theme** para todo el **proyecto**, por lo que no es necesario ponerlo para cada widget/clase Text utilizado.

24. Luego del widget **Column**, al final pero dentro del **Scaffold**, en la propiedad **FloatingActionButton** añada un **ElevatedButton** centrado, con las mismas características que el botón de la **primera pantalla**, para que se conserve la coherencia en el diseño, en la propidad **onPressed** del botón llame a un método con el nombre **_doNothing** (método **privado** por el guión bajo, que no hace nada porque es solo para poder **probar** el diseño). El código se muestra en la siguiente figura.

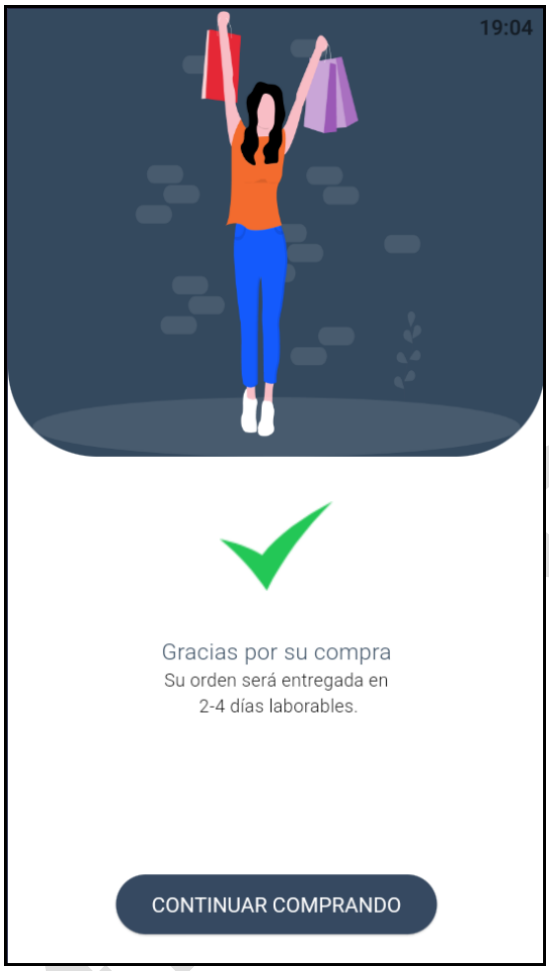


25. Para que el **error desaparezca** y poder ejecutar el proyecto, **cree** el método privado _**doNothing** al final de la clase **SuccessfulScreen**, esto se muestra a continuación.



Nota	• Se utiliza el **guion bajo** para que el método **_doNothing** sea **privado** puesto que la clase que lo contiene **SuccessfulScreen** es de alcance **público** (es decir no tiene el guion bajo, por lo tanto, es una clase **pública**). • El método **_doNothing** debe ser **privado** para ocultar su implementación de las otras clases.

26. **Ejecute** y **pruebe** el proyecto.



Clay Aldás

16/10/2026

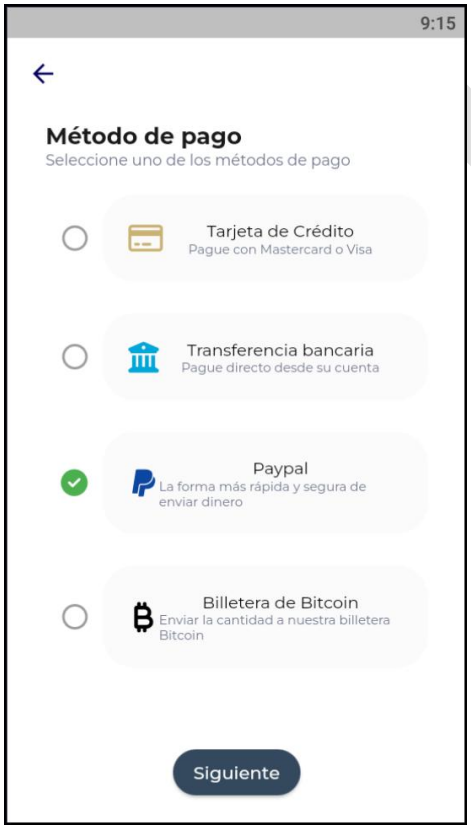
Ejercicio # 3

Ahora vamos ir realizando una **recapitulación** de lo realizado en las prácticas.

Resumen de las prácticas

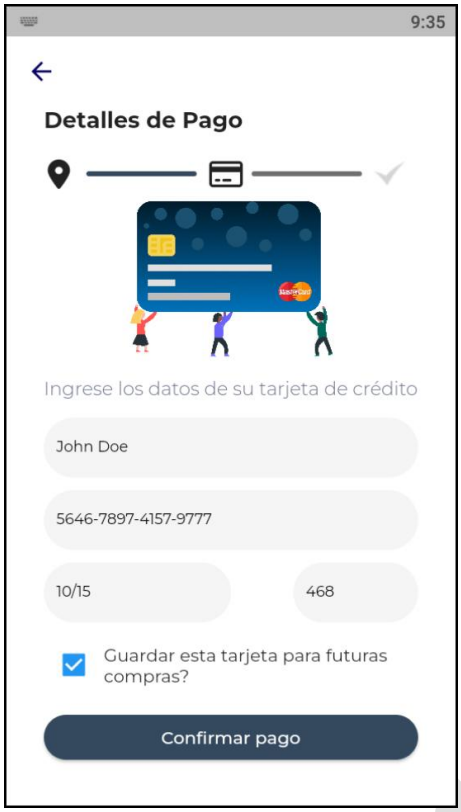
1. Hemos finalizado el diseño de la aplicación con las 3 pantallas. Ahora vamos a ir realizando una **recapitulación** de lo realizado.

2. Tenemos la **primera pantalla** con los widgets para establecer los diferentes **métodos de pago**. Pare esto se creo widgets (un widget/clase personalizado) que se comportan y parecen **radioButtons** pero en realidad no lo son.



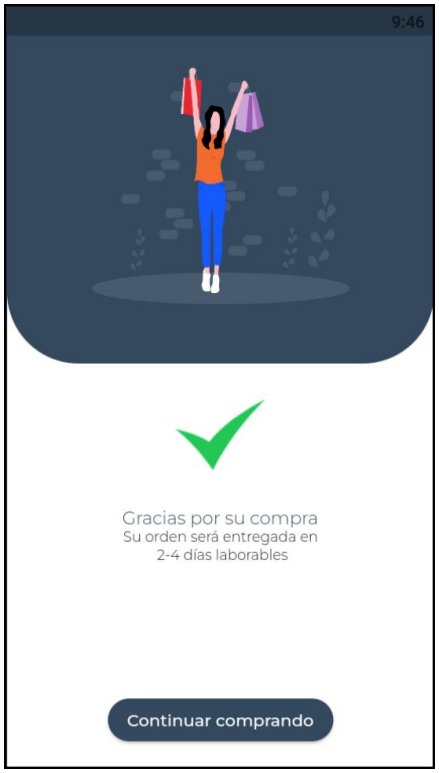
Clay Aldás

3. En la **segunda pantalla** se diseño el pago únicamente para la tarjeta de crédito, porque el resto de pagos no se lo ha realizado en las prácticas. En esta pantallla se puede **guardar los datos** de la tarjeta, para futuras compras, lo que permite optimizar el ingreso de datos.



4. En la **tercera pantalla** se ha eliminado el acceso al **historial de navegación** para que el usuario no pueda ir hacia atrás, si el usuario presiona el botón de **ir hacia atrás**, se **cierra la aplicación por seguridad**. Tambien se pasa los **argumentos** de la segunda hacia la tercera pantalla por medio de la clase **MyArguments**, aunque solo se lo imprimió en consola con el método **print** para comprobar que los parámetros se pasan correctamente, pero se lo puede utilizar en producción.

Flutter-Clay Aldás



5. Queda como práctica **propuesta** el utilizar **diferentes o adicionales widgets** a los utlizados, para crear la interfaz de usuario de la aplicación, lo que permita mejorar y optimizar el proyecto.

6. Como práctica se **propone** el utilizar cada una de las imágenes exportadas del mockup, para lo cual se utilizo **Adobe XD** y **Figma**.

7. Otro tema pendiente que se **propone** como práctica e investigación, es que las diferentes pantallas de la aplación sean **responsive**, es decir se adapten a todo tipo de tamaño de pantalllas, incluso para **tablets**.

Lecturas útiles para revisar y profundizar en los temas abordados en esta práctica.

Widget catalog

https://docs.flutter.dev/ui/widgets

Material library

https://api.flutter.dev/flutter/material/material-library.html

flutterClay Aldás